230905290 PPL LAB 8 WEEK 10

Rohan Shenoy Roll no 32 CSE D1

Q1]

1. Write a program in CUDA to add two Matrices for the following specifications:
   a. Each row of resultant matrix to be computed by one thread.
   b. Each column of resultant matrix to be computed by one thread.
   c. Each element of resultant matrix to be computed by one thread.

CODE]

```
#include <stdio.h>
#include <cuda_runtime.h>

#define ROWS 4
#define COLS 4

// kernel A
__global__ void addMatrixByRow(float *A, float *B, float *C, int rows, int
cols) {
   int row = blockIdx.x * blockDim.x + threadIdx.x;
   if (row < rows) {
       for (int col = 0; col < cols; col++) {
           int idx = row * cols + col;
           C[idx] = A[idx] + B[idx];
       }
   }
}

// kernel B
__global__ void addMatrixByCol(float *A, float *B, float *C, int rows, int
cols) {
   int col = blockIdx.x * blockDim.x + threadIdx.x;
   if (col < cols) {
       for (int row = 0; row < rows; row++) {
           int idx = row * cols + col;
           C[idx] = A[idx] + B[idx];
       }
```

```
    }
}

// kernel C
__global__ void addMatrixByElement(float *A, float *B, float *C, int rows,
int cols) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < rows && col < cols) {
        int idx = row * cols + col;
        C[idx] = A[idx] + B[idx];
    }
}


void printMatrix(float *mat, int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%.1f ", mat[i * cols + j]);
        }
        printf("\n");
    }
    printf("\n");
}

int main() {
    int size = ROWS * COLS * sizeof(float);
    float h_A[ROWS * COLS], h_B[ROWS * COLS], h_C[ROWS * COLS];

    for (int i = 0; i < ROWS * COLS; i++) {
        h_A[i] = 1.0f;
        h_B[i] = 2.0f;
    }

    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, size);
    cudaMalloc(&d_B, size);
    cudaMalloc(&d_C, size);
```

```
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    addMatrixByRow<<<1, ROWS>>>(d_A, d_B, d_C, ROWS, COLS);
    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
    printf("Result (One thread per row):\n");
    printMatrix(h_C, ROWS, COLS);

    addMatrixByCol<<<1, COLS>>>(d_A, d_B, d_C, ROWS, COLS);
    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
    printf("Result (One thread per column):\n");
    printMatrix(h_C, ROWS, COLS);

    dim3 threadsPerBlock(2, 2);
    dim3 blocksPerGrid((COLS + threadsPerBlock.x - 1) / threadsPerBlock.x,
                       (ROWS + threadsPerBlock.y - 1) / threadsPerBlock.y);
    addMatrixByElement<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C,
ROWS, COLS);
    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
    printf("Result (One thread per element):\n");
    printMatrix(h_C, ROWS, COLS);

    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    return 0;
}
```

OUTPUT]

```
6CSED1@debian:~/Desktop/ppl_lab/lab8$ nvcc q1.cu
6CSED1@debian:~/Desktop/ppl_lab/lab8$ ./a.out
Result (One thread per row):
3.0 3.0 3.0 3.0
3.0 3.0 3.0 3.0
3.0 3.0 3.0 3.0
3.0 3.0 3.0 3.0

Result (One thread per column):
3.0 3.0 3.0 3.0
3.0 3.0 3.0 3.0
3.0 3.0 3.0 3.0
3.0 3.0 3.0 3.0

Result (One thread per element):
3.0 3.0 3.0 3.0
3.0 3.0 3.0 3.0
3.0 3.0 3.0 3.0
3.0 3.0 3.0 3.0
```

Q2]

2. Write a program in CUDA to multiply two Matrices for the following specifications:
   a. Each row of resultant matrix to be computed by one thread.
   b. Each column of resultant matrix to be computed by one thread.
   c. Each element of resultant matrix to be computed by one thread.

CODE]

```
#include <stdio.h>
#include <cuda_runtime.h>

#define M 4
#define K 3
#define N 4

// kernel A
__global__ void matMulByRow(float *A, float *B, float *C, int m, int k,
int n) {
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < m) {
        for (int j = 0; j < n; j++) {
            float sum = 0;
```

```
            for (int l = 0; l < k; l++) {
                sum += A[row * k + l] * B[l * n + j];
            }
            C[row * n + j] = sum;
        }
    }
}

// kernel B
__global__ void matMulByCol(float *A, float *B, float *C, int m, int k,
int n) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (col < n) {
        for (int i = 0; i < m; i++) {
            float sum = 0;
            for (int l = 0; l < k; l++) {
                sum += A[i * k + l] * B[l * n + col];
            }
            C[i * n + col] = sum;
        }
    }
}

// kernel C
__global__ void matMulByElement(float *A, float *B, float *C, int m, int
k, int n) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < m && col < n) {
        float sum = 0;
        for (int l = 0; l < k; l++) {
            sum += A[row * k + l] * B[l * n + col];
        }
        C[row * n + col] = sum;
    }
}

void printMatrix(float *mat, int rows, int cols) {
    for (int i = 0; i < rows; i++) {
```

```
        for (int j = 0; j < cols; j++) printf("%.1f ", mat[i * cols + j]);
        printf("\n");
    }
    printf("\n");
}

int main() {
    size_t sizeA = M * K * sizeof(float);
    size_t sizeB = K * N * sizeof(float);
    size_t sizeC = M * N * sizeof(float);

    float h_A[M * K], h_B[K * N], h_C[M * N];
    for (int i = 0; i < M * K; i++) h_A[i] = 1.0f; // Matrix A of 1s
    for (int i = 0; i < K * N; i++) h_B[i] = 2.0f; // Matrix B of 2s

    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, sizeA); cudaMalloc(&d_B, sizeB); cudaMalloc(&d_C,
sizeC);
    cudaMemcpy(d_A, h_A, sizeA, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, sizeB, cudaMemcpyHostToDevice);

    matMulByRow<<<1, M>>>(d_A, d_B, d_C, M, K, N);
    cudaMemcpy(h_C, d_C, sizeC, cudaMemcpyDeviceToHost);
    printf("Method A (One thread per row):\n");
    printMatrix(h_C, M, N);

    matMulByCol<<<1, N>>>(d_A, d_B, d_C, M, K, N);
    cudaMemcpy(h_C, d_C, sizeC, cudaMemcpyDeviceToHost);
    printf("Method B (One thread per column):\n");
    printMatrix(h_C, M, N);

    dim3 threads(2, 2);
    dim3 blocks((N + 1) / 2, (M + 1) / 2);
    matMulByElement<<<blocks, threads>>>(d_A, d_B, d_C, M, K, N);
    cudaMemcpy(h_C, d_C, sizeC, cudaMemcpyDeviceToHost);
    printf("Method C (One thread per element):\n");
    printMatrix(h_C, M, N);

    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    return 0;
```

```
}
```

OUTPUT]

```
6CSED1@debian:~/Desktop/ppl_lab/lab8$ nvcc q2.cu
6CSED1@debian:~/Desktop/ppl_lab/lab8$ ./a.out
Method A (One thread per row):
6.0 6.0 6.0 6.0
6.0 6.0 6.0 6.0
6.0 6.0 6.0 6.0
6.0 6.0 6.0 6.0

Method B (One thread per column):
6.0 6.0 6.0 6.0
6.0 6.0 6.0 6.0
6.0 6.0 6.0 6.0
6.0 6.0 6.0 6.0

Method C (One thread per element):
6.0 6.0 6.0 6.0
6.0 6.0 6.0 6.0
6.0 6.0 6.0 6.0
6.0 6.0 6.0 6.0
```

ADDITIONAL QUESTIONS

Q1]

1 Write a CUDA program that reads a MXN matrix A and produces a resultant matrix B of same size as follows: Replace all the even numbered matrix elements with their row sum and odd numbered matrix elements with their column sum.

Example:

A

I/p: **1** **2** **3**
4 5 6

B

O/p: 5 **6** 9
15 7 15

CODE]

```
#include <stdio.h>
#include <cuda_runtime.h>

#define M 2
```

```
#define N 3

__global__ void transformMatrix(int *A, int *B, int m, int n) {
   int row = blockIdx.y * blockDim.y + threadIdx.y;
   int col = blockIdx.x * blockDim.x + threadIdx.x;

   if (row < m && col < n) {
       int val = A[row * n + col];
       int sum = 0;

       if (val % 2 == 0) {
           for (int j = 0; j < n; j++) {
               sum += A[row * n + j];
           }
       } else {
           for (int i = 0; i < m; i++) {
               sum += A[i * n + col];
           }
       }
       B[row * n + col] = sum;
   }
}

int main() {
   int size = M * N * sizeof(int);
   int h_A[M * N] = {1, 2, 3, 4, 5, 6};
   int h_B[M * N];

   int *d_A, *d_B;
   cudaMalloc(&d_A, size);
   cudaMalloc(&d_B, size);

   cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);

   dim3 threadsPerBlock(16, 16);
   dim3 blocksPerGrid((N + threadsPerBlock.x - 1) / threadsPerBlock.x,
                      (M + threadsPerBlock.y - 1) / threadsPerBlock.y);

   transformMatrix<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, M, N);
```

```
    cudaMemcpy(h_B, d_B, size, cudaMemcpyDeviceToHost);

    printf("Input Matrix A:\n");
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) printf("%d ", h_A[i * N + j]);
        printf("\n");
    }

    printf("\nResultant Matrix B:\n");
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) printf("%d ", h_B[i * N + j]);
        printf("\n");
    }

    cudaFree(d_A);
    cudaFree(d_B);
    return 0;
}
```

OUTPUT]

```
6CSED1@debian:~/Desktop/ppl_lab/lab8$ nvcc aq1.cu
6CSED1@debian:~/Desktop/ppl_lab/lab8$ ./a.out
Input Matrix A:
1 2 3
4 5 6

Resultant Matrix B:
5 6 9
15 7 15
6CSED1@debian:~/Desktop/ppl_lab/lab8$
```

Q2]

2 Write a CUDA program to read a matrix A of size NXN. It replaces the principal diagonal elements with zero. Elements above the principal diagonal by their factorial and elements below the principal diagonal by their sum of digits.

CODE]

```
#include <stdio.h>
#include <cuda_runtime.h>

#define N 4

__device__ int getFactorial(int n) {
    if (n <= 0) return 1;
    int res = 1;
    for (int i = 2; i <= n; i++) res *= i;
    return res;
}


__device__ int getSumOfDigits(int n) {
    int sum = 0;
    n = abs(n);
    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }
    return sum;
}


__global__ void processMatrix(int *mat, int n) {
    int i = blockIdx.y * blockDim.y + threadIdx.y;
    int j = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < n && j < n) {
        int idx = i * n + j;
        int val = mat[idx];
```

```
        if (i == j) {

            mat[idx] = 0;
        }
        else if (i < j) {

            mat[idx] = getFactorial(val);
        }
        else {

            mat[idx] = getSumOfDigits(val);
        }
    }
}

int main() {
    int size = N * N * sizeof(int);
    int h_A[N * N] = {
        5,  3,  4,  2,
        12, 6,  2,  5,
        25, 11, 7,  3,
        9,  18, 14, 8
    };
    int h_res[N * N];

    int *d_A;
    cudaMalloc(&d_A, size);
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);

    dim3 threadsPerBlock(16, 16);
    dim3 blocksPerGrid((N + 15) / 16, (N + 15) / 16);

    processMatrix<<<blocksPerGrid, threadsPerBlock>>>(d_A, N);

    cudaMemcpy(h_res, d_A, size, cudaMemcpyDeviceToHost);

    printf("Resultant Matrix:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
```

```
            printf("%4d ", h_res[i * N + j]);
        }
        printf("\n");
    }

    cudaFree(d_A);
    return 0;
}
```

OUTPUT]

```
6CSED1@debian:~/Desktop/ppl_lab/lab8$ nvcc aq2.cu
6CSED1@debian:~/Desktop/ppl_lab/lab8$ ./a.out
Resultant Matrix:
   0    6   24    2
   3    0    2  120
   7    2    0    6
   9    9    5    0
6CSED1@debian:~/Desktop/ppl_lab/lab8$
```